

GLKVM-Cloud — remote administration for the GL-iNet router fleet

Self-hosted GLKVM-Cloud (`gl-inet/glkvmm-cloud`), run as Docker containers on the consolidated services server, gives centralized remote access to all **14 GL-iNet A-1300 routers** (12 theatres + 2 VMix nodes) — their web admin UI and an SSH terminal, both reachable through one browser session, without exposing any individual router's admin interface to the WAN or needing 14 separate tunnels. Same self-hosted-over-vendor-cloud rationale already used for the DERP server and the UniFi Controller: no dependency on GL.iNet's own `glkvmm.com` cloud, no subscription, no third party in the path when administering the router fleet during a live event.

There is no physical KVM unit in this design. GLKVM-Cloud's flagship product is a KVM-over-IP hardware line (the Comet/GL-RM1) for out-of-band access to bare-metal servers — not used here. What's actually in scope is the platform's separately-documented **HTTP/HTTPS web proxy and device-management capability for embedded devices like OpenWrt and Raspberry Pi** — the A-1300s already run OpenWrt/UCI (see `config/gl-inet/`), which puts them squarely in that supported category.

Why centralize router administration at all

14 identical GL-iNet A-1300s (12 theatre routers, 2 VMix node routers) is exactly the kind of fleet that benefits from one console instead of 14 separate admin logins — same motivation as Flock for the BirdDog Play fleet (see `docs/birddog-play-rationale.md`). GLKVM-Cloud's documented features that apply directly:

- **Web-based SSH terminal** to any registered router, through the browser — no SSH client, no per-router WAN exposure, no VPN client needed on the admin's own device.
- **HTTP/HTTPS proxy to each router's own web admin (LuCI)** — same centralization for the GUI side.
- **Batch command execution across multiple devices** — push a config check or command to some or all of the 14 routers at once, rather than 14 individual sessions.
- **User group management + LDAP/OIDC** — if more than one person needs scoped admin access during the event, without sharing one shared router password.

Software: self-hosted GLKVM-Cloud — two containers

The upstream `docker-compose/docker-compose.yml` (reference) defines **two** services, both landing on the consolidated services server with their own macvlan IPs (same pattern as every other container there):

Service	Image	IP	Ports	Purpose
rttys	glzhitong/glkv-cloud:latest	192.168.1.20	443/tcp (Web UI), 5912/tcp (device connection), 10443/tcp (WebSocket proxy)	The actual GLKVM-Cloud application
coturn	coturn/coturn:edge-alpine	192.168.1.22	3478 tcp+udp	TURN relay for WebRTC (separate off-the-shelf container, not GL.iNet's own code)

`coturn` exists in the upstream stack for GLKVM-Cloud's live remote-desktop/KVM feature, which doesn't apply to routers (there's no framebuffer to capture on an OpenWrt device) — deployed anyway as the standard reference stack rather than assuming it's safe to drop; not confirmed whether `rttys`'s SSH-terminal/web-proxy features have any dependency on it internally. Revisit once actually running.

The image needs upstream's entrypoint and templates mounted. The `glzhitong/glkv-cloud` image's own entrypoint is bare `rttys` with no config; upstream's compose overrides it with a bind-mounted `docker-entrypoint.sh` that renders `rttys.conf` and `turnserver.conf` from templates using the `RTTYS_*` / `TURN_*` / `GLKVM_ACCESS_IP` variables. Without those mounts the token and password are silently ignored. This repo's compose files now do the same, mounting from an upstream checkout.

Manual Docker/docker-compose deployment supports both x86_64 and arm64 (deployment docs). Minimum self-host requirements per GL.iNet: 1 CPU core, ≥1 GB RAM, ≥40 GB storage, ≥3 Mbps — trivial next to what BirdDog Central and the rest of the stack already need on this box.

Registering the 14 routers

Each A-1300 registers to the self-hosted instance by running a connection script copied from the GLKVM-Cloud web UI (OpenWrt supports SSH/shell, so this is a normal `opkg` /script-based install, same mechanism as any other device type GLKVM-Cloud supports). Since every router already runs Tailscale and accepts routes back to `192.168.1.0/24` (see `docs/tailscale.md`), **that registration traffic rides the existing tailnet — no WAN port-forward needed for it.** WAN exposure is only about the *admin's* browser reaching the GLKVM-Cloud web UI itself from a device that isn't on the tailnet (see below).

WAN reachability — and the DERP port collision

Remote (off-venue) admin access to the GLKVM-Cloud web UI needs to be reachable from the internet, the same way `derp.example.net` is (`docs/tailscale.md`) — useful since, per this repo's own rule, only the routers and the mothership's Tailscale container are on the tailnet — no admin laptops or other client devices (see `docs/tailscale.md`). But the Cloud Gateway has one public IP, and DERP already owns WAN `443/tcp` and WAN `3478/udp` — a straight 1:1 forward of GLKVM-Cloud's own port numbers would collide with both. Resolution: forward different WAN-side port numbers to GLKVM-Cloud's real (internal, unchanged) ports —

WAN port	Proto	Forwards to	Why remapped
8443	TCP	192.168.1.20:443	WAN 443/tcp already goes to DERP
10443	TCP	192.168.1.20:10443	no collision, forwarded as-is

WAN port	Proto	Forwards to	Why remapped
3479	TCP+UDP	192.168.1.22:3478	WAN 3478/udp already goes to DERP's STUN; kept TCP alongside it on the same alternate port for one consistent "DERP = 3478, GLKVM-Cloud = 3479" mental model, even though 3478/tcp alone was actually free

Suggested hostname: `kvm.example.net` , same domain as `derp.example.net` , pointed at the same public IP — reached as `https://kvm.example.net:8443` .

What `GLKVM_ACCESS_IP` and `TURN_PORT` actually do (from upstream's `scripts/docker-entrypoint.sh` and `templates/`): `GLKVM_ACCESS_IP` is written into `rttys` 's `webrtc-ip` and `coturn` 's `external-ip` , and the auto-detect fallback only accepts an IPv4 address — so set it to the venue's **public IPv4**, not the hostname (and expect to update it if that IP isn't static). `TURN_PORT` is both the port `rttys` advertises for TURN and the port `coturn` listens on; since they're separate containers, this repo sets it to `3479` on `rttys` (the WAN-side port) and `3478` on `coturn` (where the WAN 3479 forward lands). The web UI port isn't advertised anywhere — the browser just uses `:8443` — and `10443` is forwarded unchanged.

Not fully verified against a real deployment — confirm before relying on it: the end-to-end behaviour of the `:8443` web UI and the remapped TURN port. If the remap doesn't work cleanly, the fallback is a second public IP or an SNI-routing reverse proxy in front of both DERP and GLKVM-Cloud on the literal 443 — more moving parts, only worth it if the simple remap doesn't work. Tracked in `docs/open-questions.md` (question 7).

See `config/glkvmm-cloud/` for the docker-compose template, and `config/unifi/network-config.yaml` for the port forwards above.